

Laporan TBFO

IF2224 - Teori Bahasa Formal dan Otomata



Disusun Oleh:

Geraldo Artemius - 13524005
Mikhael Andrian Yonatan - 13524051
Junior Natra Situmorang - 13524055
Reynard Nathanael - 13524103
Nicholas Luis Chandra - 13524105

DAFTAR ISI

JUDUL.....	1
DAFTAR ISI.....	2
DAFTAR GAMBAR.....	3
BAB I.....	4
1.1. Intermediate Code Generation (ICG).....	4
1.2. Control Flow Translation.....	7
1.3. Interpreter & Virtual Machine (VM).....	8
1.4. Runtime Environments.....	10
BAB II.....	12
2.1. Struktur Direktori.....	12
2.2. Implementasi Program.....	14
BAB III.....	33
3.1 Test Case 1.....	33
3.2 Test Case 2.....	37
3.3 Test Case 3.....	40
3.4 Test Case 4.....	42
3.5 Test Case 5.....	44
BAB IV.....	48
KESIMPULAN DAN SARAN.....	48
4.1. Kesimpulan.....	48
4.2. Saran.....	48
BAB V.....	49
BAB VI.....	50

DAFTAR GAMBAR

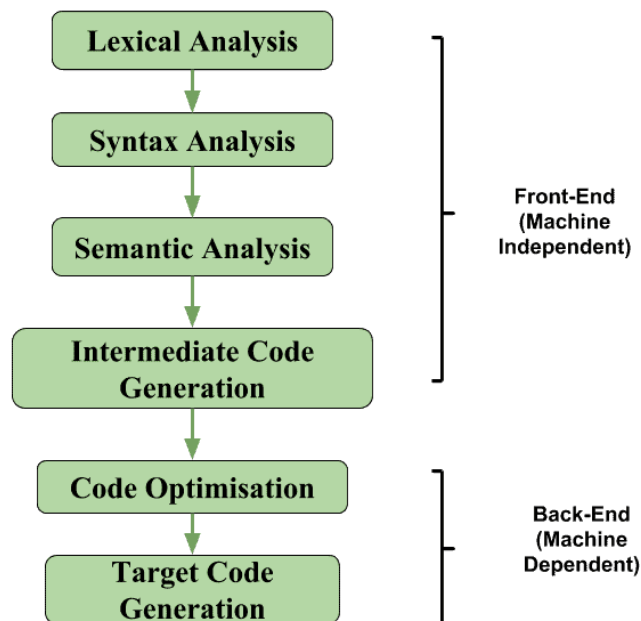
Gambar 1.1.1 Pembagian Front-end dan Back-end pada Compiler.....	4
Gambar 1.1.2 Three Address Code.....	6
Gambar 1.2.1 Visualisasi Control Flow Translation.....	7
Gambar 1.3.1 Fetch, Decode, Execute Cycle.....	8
Gambar 1.3.2 Activation Record yang Dialokasikan VM.....	9
Gambar 1.4.1 Storage Allocation.....	10

BAB I

Landasan Teori

1.1. Intermediate Code Generation (ICG)

Intermediate Code Generation (ICG) adalah fase yang menjadi penengah antara fase analisis (*front-end*) dan fase sintesis (*back-end*). Fase *front-end* (analisis leksikal, sintaksis, dan semantik yang bertanggung jawab untuk memeriksa validitas dari kode sumber dan menghasilkan representasi internal seperti Abstract Syntax Tree (AST). ICG kemudian mengambil representasi tersebut dan menerjemahkannya menjadi kode perantara (Intermediate Code) yang bersifat linier dan independen terhadap arsitektur perangkat keras (machine-independent).



Gambar 1.1.1 Pembagian Front-end dan Back-end pada Compiler

(Sumber: <https://www.geeksforgeeks.org/compiler-design/intermediate-code-generation-in-compiler-design/>)

Kode perantara yang dihasilkan oleh ICG nantinya memberikan dua keuntungan utama dalam rekayasa perangkat lunak kompilator. Keuntungan utama dari adanya kode perantara yaitu:

- Kemudahan Portabilitas (Retargeting)

Misalkan suatu kompilator mendukung M bahasa pemrograman tingkat tinggi dan N jenis arsitektur mesin/prosesor yang berbeda. Tanpa adanya kode perantara maka perlu dibangun MxN Kompilator yang terpisah. Dengan adanya IC, keperluan tersebut berubah menjadi M+N Modul yang perlu dibuat.

- Optimasi Independen-Mesin (Machine-Independent Optimization)
Dengan adanya kode perantara dapat meningkatkan efisiensi eksekusi atau menghemat penggunaan memori. Karena IC bersifat universal dan tidak terikat pada register atau instruksi spesifik perangkat keras tertentu, algoritma optimasi yang sama dapat diterapkan pada target mesin manapun.

Representasi kode perantara dapat dikategorikan menjadi dua kategori utama berdasarkan strukturnya. Kedua kategorinya adalah sebagai berikut.

- Representasi Grafis (Graphical Representation)
Salah dua Representasi Grafis dapat berupa Syntax Tree atau Directed Acyclic Graph (DAG). Representasi ini mempertahankan hierarki struktur program, namun sulit untuk dieksekusi secara langsung oleh mesin sekuensial.
- Representasi Linier (Linear Representation)
Representasi Linier dapat berupa Postfix Notation dan Three-Address Code. Kategori representasi ini meratakan struktur grafis menjadi barisan instruksi berurutan baris demi baris, sehingga sangat ramah terhadap mekanisme pembacaan memori komputer.

TAC (Three-Address Code) adalah salah satu bentuk representasi yang linier paling populer dalam perancangan kompilator. Karakteristik utama dari TAC adalah bahwa setiap instruksi didalamnya maksimal hanya memiliki tiga alamat memori (atau referensi). Format umum dari instruksi TAC adalah $x = y \oplus z$.

Dimana x , y , dan z adalah alamat berupa variabel lokal, variable global, atau variabel sementara/temporary variable yang diciptakan oleh kompilator, sedangkan $\oplus$ melambangkan operator aritmatika atau logika. Jika sebuah ekspresi tingkat tinggi memiliki operasi yang kompleks, kompilator akan memecahnya menjadi beberapa baris instruksi TAC dasar menggunakan variabel bantuan (t_1 , t_2 , ...).

Implementasi struktur data TAC dapat diwujudkan menggunakan struktur data record atau tabel. Terdapat tiga skema implementasi utama untuk merepresentasikan TAC.

#	Op	Arg1	Arg2	Result
(1)	+	x	y	t1
(2)	+	y	z	t2
(3)	*	t1	t2	t3
(4)	+	t1	z	t4
(5)	+	t3	t4	t5

Quadruple representation

#	Op	Arg1	Arg2
(1)	+	x	y
(2)	+	y	z
(3)	*	(1)	(2)
(4)	+	(1)	z
(5)	+	(3)	(4)

Triples representation

#	Op	Arg1	Arg2
(14)	+	x	y
(15)	+	y	z
(16)	*	(14)	(15)
(17)	+	(14)	z
(18)	+	(16)	(17)

List of pointers to table

#	Statement
(1)	(14)
(2)	(15)
(3)	(16)
(4)	(17)
(5)	(18)

Indirect Triples representation

Gambar 1.1.2 Three Address Code

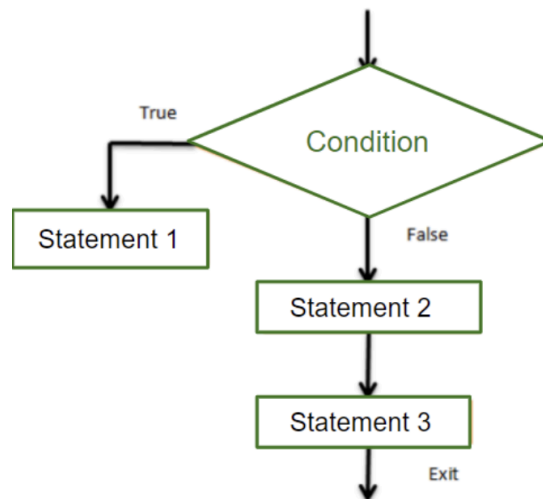
(Sumber: <https://www.geeksforgeeks.org/compiler-design/three-address-code-compiler/>)

- **Quadruples**
Menggunakan struktur data dengan empat field (bidang): op (operator), arg1 (argumen 1), arg2 (argumen 2), dan result (hasil). Variabel sementara secara eksplisit ditulis pada kolom result.
- **Triples**
Menggunakan struktur data dengan tiga field: op, arg1, dan arg2. Pendekatan ini tidak menggunakan nama variabel sementara secara eksplisit untuk menampung hasil. Sebagai gantinya, hasil dari suatu operasi direferensikan langsung menggunakan indeks baris posisi instruksi tersebut berada (misalnya (0) atau (1)).
- **Indirect Triples**
Merupakan variasi dari skema Triples, di mana kompilator menggunakan sebuah array penunjuk (pointer) terpisah untuk mendaftarkan urutan eksekusi operasi. Skema ini sangat menguntungkan saat fase optimasi, karena kompilator cukup menggeser indeks pada array penunjuk tanpa harus memindahkan struktur data instruksi utamanya di memori.

1.2. Control Flow Translation

Secara default, suatu program dieksekusi secara berurutan dari baris pertama hingga baris terakhir, pola ini dikenal dengan sebutan *sequential execution*. Namun, program yang lebih kompleks tidak selalu dapat diselesaikan hanya dengan eksekusi sekuensial. Untuk itu, bahasa pemrograman menyediakan control flow statement konstruksi yang mengatur arah eksekusi berdasarkan kondisi, pengulangan, atau lompatan alur tertentu.

Secara umum, control flow dapat dibagi menjadi tiga kategori utama, yaitu *selection statement*, *iteration statement*, dan *jump statement*. *Selection statement* digunakan untuk memilih blok instruksi tertentu berdasarkan hasil evaluasi kondisi, seperti *if*, *if-else*, atau *switch*. *Iteration statement* digunakan untuk menjalankan blok instruksi secara berulang selama kondisi tertentu masih terpenuhi, seperti *while*, *for*, atau *do-while*. Terakhir, *jump statement* digunakan untuk memindahkan eksekusi secara langsung ke bagian lain dari program, seperti *break*, *continue*, *return*, atau *goto*.



Gambar 1.2.1 Visualisasi Control Flow Translation

(Sumber: <https://www.geeksforgeeks.org/compiler-design/backpatching-in-compiler-design/>)

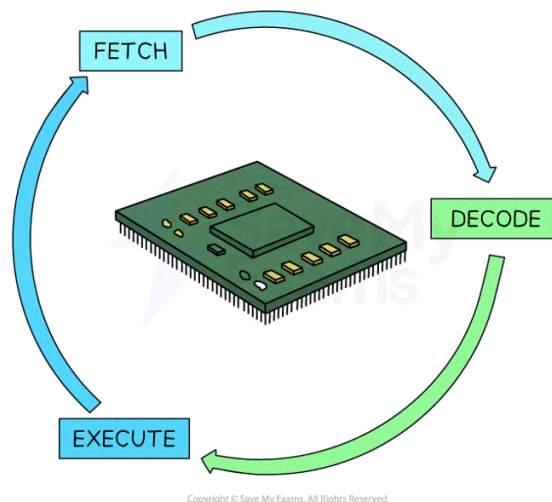
Dalam konteks compiler dan interpreter, struktur kontrol tingkat tinggi ini tidak dapat langsung dieksekusi dalam bentuk sintaks asli bahasa pemrograman. Struktur seperti *if-else* dan *while* umumnya masih dalam bentuk *nested* di dalam *AST* atau *decorated AST*, artinya, blok-blok instruksi tersebut saling bersarang membentuk hierarki pohon. Agar dapat dieksekusi oleh interpreter, struktur ini perlu diterjemahkan menjadi deretan instruksi yang *flat* dan sekuensial. Proses inilah yang disebut sebagai *control flow translation*.

Control flow translation bertujuan mengubah struktur kontrol yang hierarkis menjadi daftar instruksi sekuensial yang tetap mampu merepresentasikan percabangan dan pengulangan. Dalam *intermediate code*, mekanisme ini umumnya diimplementasikan menggunakan label dan *jump*. Label berfungsi sebagai penanda posisi suatu instruksi, sedangkan *jump* berfungsi

memindahkan eksekusi ke label tersebut. Dengan demikian, meskipun intermediate code berbentuk linear, alur eksekusinya tetap dapat berpindah sesuai logika program.

1.3. Interpreter & Virtual Machine (VM)

Dalam fase sintesis pada perancangan kompilator, setelah tahapan pembangkitan kode antara (Intermediate Code Generator) menghasilkan representasi linear seperti Three-Address Code (TAC), program memerlukan mesin eksekusi untuk dapat berjalan. Pada titik ini, sebuah Interpreter berfungsi sebagai Virtual Machine (VM) yang mensimulasikan cara kerja perangkat keras komputer dalam lingkungan perangkat lunak. VM ini tidak mengeksekusi kode mesin secara langsung, melainkan mengeksekusi instruksi TAC secara berurutan menggunakan penunjuk instruksi yang secara konseptual dikenal sebagai Program Counter (PC) atau Instruction Pointer (IP).



Gambar 1.3.1 *Fetch, Decode, Execute Cycle*

(Sumber:

<https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute-cycle/>)

VM beroperasi dengan siklus eksekusi perangkat keras yang diadaptasi ke dalam perangkat lunak, yaitu :

- *Fetch* (Pengambilan): Mengambil instruksi operasional berikutnya dari memori program berdasarkan alamat yang saat ini ditunjuk oleh Instruction Pointer.
- *Decode* (Dekode): Menerjemahkan instruksi tersebut untuk mengidentifikasi jenis operasi (seperti operasi aritmatika, assignment, atau percabangan) beserta operan-operan yang terlibat di dalamnya.
- *Execute* (Eksekusi): Melakukan aksi komputasi yang diminta oleh instruksi. Setelah eksekusi berhasil, Instruction Pointer akan diperbarui secara otomatis (biasanya bergerak

maju sebesar satu langkah) untuk menunjuk ke instruksi berikutnya, kecuali instruksi tersebut adalah perintah lompatan (jump/branch) yang mengharuskan kontrol program berpindah ke label atau baris tertentu.

Selama masa eksekusi program (runtime), interpreter memerlukan alokasi ruang memori dinamis untuk menyimpan status program, mengevaluasi ekspresi matematis, dan mengelola alur kontrol. Struktur data utama yang mengorkestrasi memori ini adalah *Stack* (tumpukan), yang beroperasi dengan prinsip LIFO (*Last In, First Out*). Penggunaan *stack* jauh lebih efektif dibandingkan penyimpanan dalam bentuk *array* datar (*linear*) karena sifat eksekusi program sangat bergantung pada konsep ruang lingkup variabel (scope) dan hierarki pemanggilan fungsi (*function call*).

0	1	2	3	4	5	6	7	8	9
Param 1	...		...	Param n	Local 1	...		...	Local m

Activation record for a static method with n parameters and m locals.

0	1	2	3	4	5	6	7	8	9	10
this	Param 1	...		...	Param n	Local 1	...		...	Local m

Activation record for a non-static method with n parameters and m locals.

Gambar 1.3.2 *Activation Record* yang Dialokasikan VM

(Sumber:

<https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute-cycle/>)

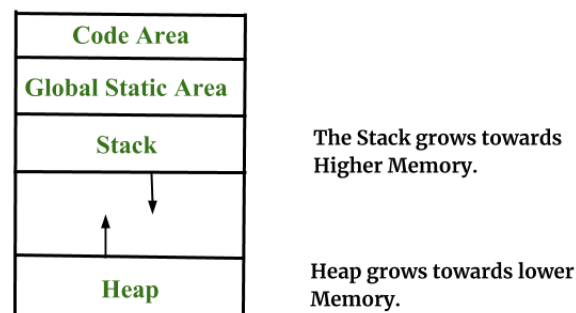
Setiap kali sebuah subprogram atau blok kode baru dieksekusi, VM akan mengalokasikan sebuah blok memori khusus di dalam tumpukan yang disebut *Activation Record* (atau *Stack Frame*). Sebuah *activation record* mengenkapsulasi seluruh informasi untuk mengeksekusi satu instansiasi fungsi, termasuk parameter fungsi yang dikirim (*arguments*), variabel lokal, nilai sementara (*temporaries*), dan alamat pengembalian (*return address*). Ketika fungsi dipanggil, *activation record* baru akan di-*push* ke atas tumpukan, dan interpreter menggeser fokus operasinya secara eksklusif ke tumpukan paling atas untuk menjamin isolasi variabel lokal dari fungsi lain. Setelah fungsi selesai dieksekusi, *activation record* tersebut akan di-*pop* dari *stack*. Mekanisme ini memastikan memori dibebaskan secara otomatis saat keluar dari fungsi dan mencegah intervensi memori (*memory leak*) antarfungsi.

Meskipun tahapan Analisis Semantik (Semantic Analysis) telah memvalidasi tipe secara statis (static type checking) sebelum program berjalan, validasi saat runtime (dinamis) menyediakan lapisan keamanan memori tambahan. Pengecekan statis mengevaluasi aturan

struktural kode, sementara pengecekan dinamis berhadapan langsung dengan nilai aktual yang bersarang di dalam memori eksekusi.

Validasi tipe dinamis memverifikasi kesesuaian tipe dari nilai aktual di memori dengan memanfaatkan informasi tipe bawaan yang diturunkan dari *Decorated Abstract Syntax Tree* (AST). Mekanisme ini sangat esensial pada bahasa pemrograman *weakly-typed* atau dinamis, di mana sebuah nilai bisa diubah secara implisit yang dapat memicu ambiguitas eksekusi. Sebelum Interpreter menjalankan operasi (*Execute*) seperti penambahan aritmatika, VM mencocokkan tipe data operan yang tersimpan dalam *stack* dengan spesifikasi pada *Decorated AST*. Apabila ditemukan ketidakcocokan tipe saat program berjalan (misalnya membaca nilai string di tempat yang seharusnya integer akibat manipulasi masukan), interpreter akan menghentikan eksekusi dan memicu *Runtime Error*. Pada bahasa *strongly-typed*, meski aturan sudah ketat secara statis, mekanisme ini menangguhkan anomali saat *runtime* yang tidak bisa ditebak oleh kompilator.

1.4. Runtime Environments



Gambar 1.4.1 *Storage Allocation*

(Sumber: <https://www.geeksforgeeks.org/compiler-design/runtime-environments-in-compiler-design/>)

Runtime Environments merupakan sistem yang memfasilitasi dan mendukung jalannya sebuah program yang telah dikompilasi. Sistem ini bertindak sebagai jembatan yang menghubungkan program statis dengan aksi komputasi dinamis yang terjadi pada saat eksekusi perangkat keras. Tanggung jawab utama dari *runtime environments* seperti pemetaan pengidentifikasi ke lokasi memori aktual, manajemen pemanggilan prosedur, mekanisme pengiriman parameter, serta penyediaan layanan esensial lainnya seperti manajemen alur kontrol dan akses ke pustaka.

Pengelolaan siklus hidup data selama program berjalan, memori eksekusi dibagi menjadi beberapa wilayah, yaitu *Target Code* (bersifat statis dan berisi instruksi-instruksi program hasil kompilasi), *Static Data* (menyimpan variabel global dan variabel statis yang masa hidupnya (*lifetime*) berlangsung sepanjang eksekusi program, *Stack* (bersifat dinamis dan mengelola

variabel otomatis dan *activation records*), dan *Heap* (bersifat dinamis dan digunakan untuk mengelola alokasi memori dinamis yang ukurannya bisa berubah-ubah saat eksekusi).

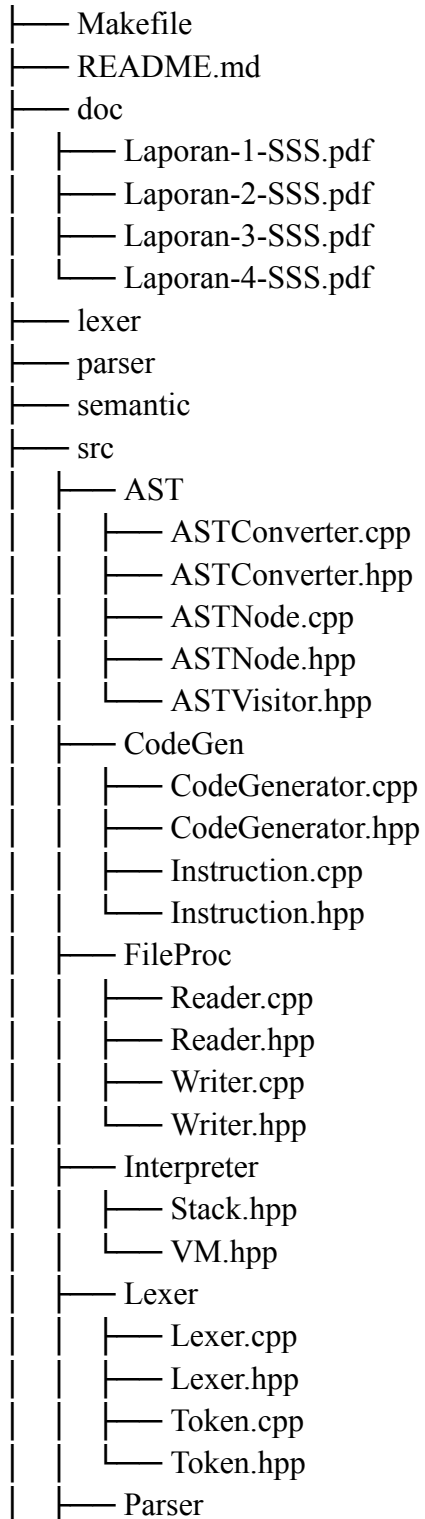
Dalam mendistribusikan ruang memori, kompilator menerapkan beberapa strategi alokasi utama yang disesuaikan dengan kebutuhan struktur data. Alokasi Statis (*Static Storage Allocation*) mengikat variabel pada lokasi memori tetap sejak awal kompilasi, namun metode ini memiliki keterbatasan karena tidak dapat mendukung pemanggilan fungsi secara rekursif. Untuk mendukung rekursi, digunakan Alokasi Stack (*Stack Storage Allocation*) yang memberikan ruang memori baru pada setiap pemanggilan prosedur dan otomatis membebaskannya setelah selesai dieksekusi. Sementara itu, untuk data dinamis yang ukurannya tidak terprediksi dan siklus hidupnya bebas dari alur prosedur seperti *linked list* dan *tree* sistem menggunakan Alokasi Heap (*Heap Storage Allocation*) yang memungkinkan memori dialokasikan dan didealokasikan kapan saja.

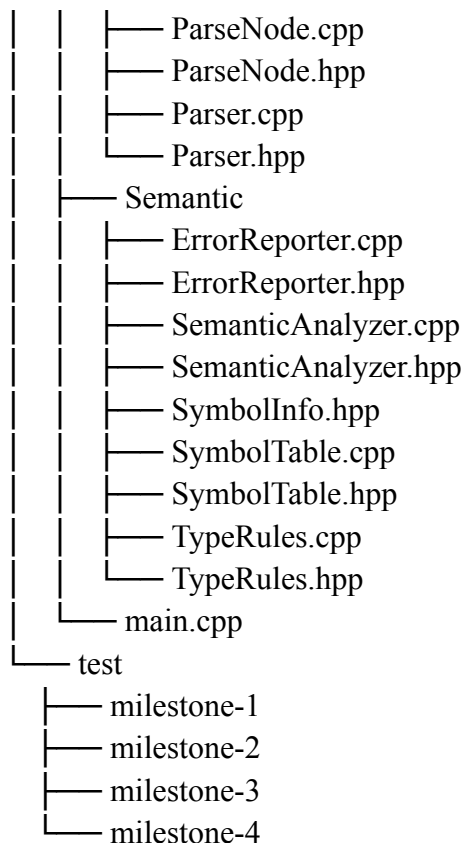
Aspek krusial terakhir yang dikelola oleh lingkungan eksekusi adalah mekanisme pengiriman parameter (*Parameter Passing*), yaitu metode untuk mentransfer data dari prosedur pemanggil ke prosedur yang dipanggil. Pendekatan yang paling umum adalah *Call by Value*, di mana sistem menyalin nilai dari argumen aktual ke parameter formal di dalam *activation record* baru, sehingga modifikasi di dalam fungsi tidak akan memengaruhi variabel aslinya. Pendekatan lainnya adalah *Call by Reference*, yang mengirimkan alamat memori (L-value) dari parameter aktual ke dalam fungsi. Melalui metode referensi ini, segala bentuk perubahan yang terjadi di dalam prosedur akan langsung berdampak dan memodifikasi variabel asli di memori pemanggil.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Struktur Direktori





Berikut struktur folder kami, masih sama dengan laporan sebelumnya, secara garis besar :

- Folder doc berisi dokumen laporan Tugas Besar dalam bentuk dokumen PDF.
- Folder src berisi *source code* program dalam bahasa C++ yang mencakup implementasi mesin DFA (Lexer.cpp, Lexer.hpp) pada folder Lexer, definisi token (Token.cpp, Token.hpp) pada folder Lexer juga, pemrosesan file (Reader.cpp, Reader.hpp, Writer.cpp, Writer.hpp) pada folder FileProc, integrasi seluruh modul (main.cpp), implementasi modul *parse tree* pada folder Parser (ParseNode.hpp, ParseNode.cpp, Parser.cpp, dan Parser.hpp). Selain itu, saat ini program juga telah diperluas dengan penambahan dua folder baru, yaitu folder AST yang memuat struktur *Abstract Syntax Tree* beserta *visitor* dan konverternya (ASTNode.hpp, ASTNode.cpp, ASTConverter.hpp, ASTConverter.cpp, ASTVisitor.hpp), serta folder Semantic yang menangani tahapan analisis semantik (TypeRules.hpp, TypeRules.cpp, SymbolInfo.hpp, SymbolTable.cpp, SemanticAnalyzer.hpp, SemanticAnalyzer.cpp, ErrorReporter.hpp, ErrorReporter.cpp). Untuk tahap selanjutnya, terdapat folder CodeGen yang berisi generator instruksi/p-code (CodeGenerator.cpp, CodeGenerator.hpp, Instruction.cpp, Instruction.hpp) dan folder Interpreter untuk mesin virtual eksekusi (VM.cpp, VM.hpp, Stack.cpp, Stack.hpp).

- Folder test berisi file pengujian, baik berupa data masukan (source code bahasa Arion) maupun data keluaran hasil pemrosesan token. Terdapat folder milestone-1, milestone-2, milestone-3, milestone-4 di dalamnya untuk menentukan *test case* per *milestone*.
- File Makefile berisi kumpulan instruksi kompilasi untuk mempermudah proses build source code C++ menjadi program executable secara otomatis.

2.2. Implementasi Program

1. Instruction.hpp & Instruction.cpp

Instruction.hpp

```
enum class PCodeOp {
    LIT,
    LOD,
    STO,
    CAL,
    INT,
    JMP,
    JPC,
    OPR,
    RET,
    LODA,
    LODI,
    STOI,
    CHK
};

enum class OprCode {
    NEG = 1,
    ADD = 2,
    SUB = 3,
    MUL = 4,
    DIV = 5,
    MOD = 6,
    EQL = 7,
    NEQ = 8,
    LSS = 9,
    GEQ = 10,
    GTR = 11,
    LEQ = 12,
    WRT = 13,
    WRTLN = 14,
    AND = 15,
    OR = 16,
    WRTBOOL = 17
};
```

```

struct Instruction {
    PCodeOp op = PCodeOp::LIT;
    int level = 0;
    int value = 0;
    std::string text;
    bool hasText = false;

    Instruction() = default;
    Instruction(PCodeOp op, int level, int value);
    Instruction(PCodeOp op, const std::string& text);
};

std::string toString(PCodeOp op);
std::string toString(OprCode op);
std::string toString(const Instruction& instr);

std::ostream& operator<<(std::ostream& os, const Instruction& instr);

```

Bagian ini mendefinisikan representasi instruksi P-Code untuk tahap code generation dan eksekusi VM. Di dalamnya ada dua enum utama, PCodeOp untuk jenis instruksi, serta OprCode untuk operasi aritmatika/logika yang dipakai saat PCodeOp::OPR. Struktur Instruction menyimpan satu instruksi dengan atribut op (opcode), level (level scope/aktivasi), value (operand numerik), text (operand string opsional), dan hasText (penanda apakah text digunakan). Disediakan dua konstruktor: satu untuk instruksi numerik (op, level, value) dan satu untuk instruksi yang membawa teks (op, text). Selain itu ada fungsi toString() untuk mengubah PCodeOp, OprCode, atau Instruction menjadi string, serta overload operator<< agar Instruction bisa langsung dicetak ke stream.

Instruction.cpp

```

Instruction::Instruction(PCodeOp op, int level, int value){
    INPUT : PCodeOp opcode, integer level, integer value.
    OUTPUT : Objek Instruction dengan operand numerik.
    CARA KERJA :
    1. Menyalin op ke atribut opcode.
    2. Menyimpan level dan value sebagai operand numerik.
    3. Mengatur hasText menjadi false karena tidak memakai teks.
}

Instruction::Instruction(PCodeOp op, const std::string& text){
    INPUT : PCodeOp opcode, referensi string text.
    OUTPUT : Objek Instruction dengan operand teks.
    CARA KERJA :

```

```

1. Menyalin op ke atribut opcode.
2. Menginisialisasi level dan value dengan 0 sebagai default.
3. Menyimpan text ke atribut teks.
4. Mengatur hasText menjadi true karena m
}

std::string toString(PCodeOp op){
    INPUT : PCodeOp opcode.
    OUTPUT : String nama opcode.
    CARA KERJA :
    1. Melakukan switch pada nilai op.
    2. Mengembalikan string literal sesuai opcode (misalkan LIT, LOD,
        JMP).
    3. Jika tidak ada yang cocok, mengembalikan string kosong.
}

std::string toString(OprCode op){
    INPUT : OprCode opcode.
    OUTPUT : String nama operasi.
    CARA KERJA :
    1. Melakukan switch pada nilai op.
    2. Mengembalikan string literal sesuai operasi (misalkan ADD, SUB,
        EQL, WRTLN).
    3. Jika tidak ada yang cocok, mengembalikan "WRT" sebagai fallback.
}

std::string toString(const Instruction& instr){
    INPUT : Referensi Instruction yang akan diformat.
    OUTPUT : String representasi instruksi.
    CARA KERJA :
    1. Membuat ostream sebagai buffer keluaran.
    2. Jika opcode RET, hanya menulis "RET" tanpa operand.
    3. Selain itu, menulis opcode level value menggunakan
        toString(PCodeOp).
    4. Jika hasText bernilai true, menambahkan ';' di belakang
        instruksi.
    5. Mengembalikan string hasil format.
}

std::ostream& operator<<(std::ostream& os, const Instruction& instr){
    INPUT : Referensi stream output dan referensi Instruction.
    OUTPUT : Stream yang sudah berisi string instruksi.
    CARA KERJA :
    1. Memanggil toString(instr) untuk mendapatkan format string
        instruksi.
    2. Menulis hasilnya ke os.
    3. Mengembalikan os agar chaining << tetap bisa digunakan.
}

```


2. CodeGenerator.hpp & CodeGenerator.cpp

CodeGenerator.hpp

```
class CodeGenerator : public ASTVisitor {
private:
    SymbolTable* symbolTable;
    std::vector<Instruction> instructions;
    int currentLevel = 0;
    bool isLValueMode = false;
    bool isAddressMode = false;
    void emit(const Instruction& instr);
    int resolveTabIndex(ASTNode* node) const;
    int resolveLevel(ASTNode* node) const;
    int levelDiff(ASTNode* node) const;

    OprCode mapBinaryOpr(const std::string& op) const;
    OprCode mapUnaryOpr(const std::string& op) const;
    bool isRelationalOp(const std::string& op) const;

public:
    explicit CodeGenerator(SymbolTable& table);

    void generate(ASTNode* root);
    const std::vector<Instruction>& getInstructions() const;
    std::string dump() const;

    void visitProgramNode(ProgramNode* node) override;
    void visitCompoundNode(CompoundNode* node) override;
    void visitConstDeclNode(ConstDeclNode* node) override;
    void visitTypeDeclNode(TypeDeclNode* node) override;
    void visitVarDeclNode(VarDeclNode* node) override;
    void visitParamNode(ParamNode* node) override;
    void visitSubprogramDeclNode(SubprogramDeclNode* node) override;

    void visitNamedTypeNode(NamedTypeNode* node) override;
    void visitArrayTypeNode(ArrayTypeNode* node) override;
    void visitRecordTypeNode(RecordTypeNode* node) override;
    void visitRangeNode(RangeNode* node) override;
    void visitEnumNode(EnumNode* node) override;

    void visitAssignNode(AssignNode* node) override;
    void visitIfNode(IfNode* node) override;
    void visitCaseBlockNode(CaseBlockNode* node) override;
    void visitCaseNode(CaseNode* node) override;
    void visitWhileNode(WhileNode* node) override;
    void visitRepeatNode(RepeatNode* node) override;
    void visitForNode(ForNode* node) override;
```

```

void visitProcCallNode(ProcCallNode* node) override;

void visitVarNode(VarNode* node) override;
void visitArrayAccessNode(ArrayAccessNode* node) override;
void visitRecordAccessNode(RecordAccessNode* node) override;

void visitBinOpNode(BinOpNode* node) override;
void visitUnaryOpNode(UnaryOpNode* node) override;
void visitNumberNode(NumberNode* node) override;
void visitStringNode(StringNode* node) override;
void visitCharNode(CharNode* node) override;
};

```

Bagian ini mendefinisikan kelas CodeGenerator sebagai ASTVisitor untuk menghasilkan P-Code dari AST. Atribut utamanya seperti symbolTable sebagai referensi symbol table, instructions sebagai daftar Instruction, currentLevel untuk level scope aktif, dan isLValueMode untuk mode akses l-value. Fungsi-fungsi generate() memulai proses codegen, getInstructions() mengembalikan daftar instruksi, dump() menghasilkan representasi teks, serta fungsi helper seperti emit(), resolveTabIndex(), resolveLevel(), dan mapping operator (mapBinaryOpr, mapUnaryOpr). Metode visit* mengimplementasikan proses codegen untuk setiap jenis node AST.

CodeGenerator.cpp

```

std::string stripQuotedLiteral(const std::string& value){
    INPUT : String literal bertanda kutip.
    OUTPUT : String tanpa kutip jika valid.
    CARA KERJA :
    1. Mengecek panjang string minimal 2.
    2. Mengecek pasangan kutip di awal dan akhir.
    3. Jika cocok, mengembalikan substring tanpa kutip.
    4. Jika tidak cocok, mengembalikan string asli.
}

CodeGenerator::CodeGenerator(SymbolTable& table){
    INPUT : Referensi SymbolTable.
    OUTPUT : Objek CodeGenerator terinisialisasi.
    CARA KERJA :
    1. Menyimpan alamat table ke symbolTable.
}

void CodeGenerator::emit(const Instruction& instr){
    INPUT : Instruction yang akan disimpan.
    OUTPUT : Instruksi ditambahkan ke daftar.
    CARA KERJA :
    1. Menambahkan instr ke instructions.
}

```

```

}

int CodeGenerator::resolveTabIndex(ASTNode* node){
    INPUT : Pointer node AST.
    OUTPUT : Nilai tabIndex atau -1.
    CARA KERJA :
    1. Jika node null, kembalikan -1.
    2. Kembalikan node→tabIndex.
}

int CodeGenerator::resolveLevel(ASTNode* node){
    INPUT : Pointer node AST.
    OUTPUT : Nilai lev atau -1.
    CARA KERJA :
    1. Jika node null, kembalikan -1.
    2. Kembalikan node→lev.
}

int CodeGenerator::levelDiff(ASTNode* node){
    INPUT : Pointer node AST.
    OUTPUT : Selisih level dengan currentLevel.
    CARA KERJA :
    1. Ambil level deklarasi node.
    2. Jika tidak valid, kembalikan 0.
    3. Hitung currentLevel - declLevel.
}

OprCode CodeGenerator::mapBinaryOpr(const std::string& op){
    INPUT : String operator biner.
    OUTPUT : OprCode yang sesuai.
    CARA KERJA :
    1. Membandingkan op dengan operator yang didukung.
    2. Mengembalikan OprCode yang sesuai.
    3. Jika tidak ditemukan, fallback ke ADD.
}

OprCode CodeGenerator::mapUnaryOpr(const std::string& op){
    INPUT : String operator unary.
    OUTPUT : OprCode unary.
    CARA KERJA :
    1. Jika op adalah -, kembalikan NEG.
    2. Selain itu, fallback ke NEG.
}

bool CodeGenerator::isRelationalOp(const std::string& op){
    INPUT : String operator.
    OUTPUT : Boolean apakah operator relasional.
    CARA KERJA :
    1. Mengecek apakah op berada di himpunan relasional.

```

```

    2. Mengembalikan hasil pengecekan.
}

void CodeGenerator::generate(ASTNode* root){
    INPUT : Pointer root AST.
    OUTPUT : Instruksi P-Code tersimpan di instructions.
    CARA KERJA :
    1. Jika root tidak null, panggil accept(this).
}

const std::vector<Instruction>& CodeGenerator::getInstructions()
const{
    INPUT : -
    OUTPUT : Referensi list instruksi.
    CARA KERJA :
    1. Mengembalikan instructions.
}

std::string CodeGenerator::dump() const {
    INPUT : -
    OUTPUT : String seluruh instruksi.
    CARA KERJA :
    1. Loop seluruh instruksi.
    2. Menuliskan indeks dan instruksi per baris.
    3. Mengembalikan hasil string.
}

void CodeGenerator::visitProgramNode(ProgramNode* node){
    INPUT : Node program.
    OUTPUT : Instruksi setup dan main.
    CARA KERJA :
    1. Set currentLevel = 0.
    2. Hitung total variabel global.
    3. Emit INT untuk alokasi.
    4. Proses deklarasi dan main block.
    5. Emit RET sebagai akhir.
}

void CodeGenerator::visitCompoundNode(CompoundNode* node){
    INPUT : Node blok.
    OUTPUT : Instruksi untuk setiap statement.
    CARA KERJA :
    1. Iterasi seluruh statement.
    2. Panggil accept per statement.
}

void CodeGenerator::visitConstDeclNode(ConstDeclNode* node){
    INPUT : Node konstanta.
    OUTPUT : Tidak menghasilkan instruksi.
}

```

```

    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitTypeDeclNode(TypeDeclNode* node){
    INPUT : Node tipe.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitVarDeclNode(VarDeclNode* node){
    INPUT : Node var.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitParamNode(ParamNode* node){
    INPUT : Node parameter.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitSubprogramDeclNode(SubprogramDeclNode* node)
{
    INPUT : Node subprogram.
    OUTPUT : Instruksi fungsi/prosedur.
    CARA KERJA :
    1. Simpan indeks JMP untuk skip.
    2. Catat entryAddress di symbol table.
    3. Naikkan currentLevel.
    4. Hitung local var dan param.
    5. Emit INT alokasi.
    6. Proses deklarasi dan body.
    7. Emit RET.
    8. Pulihkan level.
    9. Patch alamat JMP.
}

void CodeGenerator::visitNamedTypeNode(NamedTypeNode* node) {
    INPUT : Node tipe bernama.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

```

```

void CodeGenerator::visitArrayTypeNode(ArrayTypeNode* node){
    INPUT : Node array.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitRecordTypeNode(RecordTypeNode* node){
    INPUT : Node record.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitRangeNode(RangeNode* node){
    INPUT : Node range.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

CodeGenerator::visitEnumNode(EnumNode* node){
    INPUT : Node enum.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitAssignNode(AssignNode* node){
    INPUT : Node assignment.
    OUTPUT : Instruksi store nilai.
    CARA KERJA :
    1. Generate nilai RHS.
    2. Jika target VarNode, emit ST0.
    3. Jika belum valid, lookup symbol table lalu emit ST0.
    4. Jika bukan var langsung, set isLValueMode, kunjungi target.
}

void CodeGenerator::visitIfNode(IfNode* node){
    INPUT : Node if.
    OUTPUT : Instruksi branch.
    CARA KERJA :
    1. Generate kondisi.
    2. Emit JPC placeholder.
    3. Generate then-block.
    4. Jika ada else, emit JMP placeholder.
    5. Patch alamat else.
    6. Generate else dan patch akhir.
}

```

```

}

void CodeGenerator::visitCaseBlockNode(CaseBlockNode* node){
    INPUT : Node case block.
    OUTPUT : Instruksi statement.
    CARA KERJA :
    1. Jika ada statement, panggil accept.
}

void CodeGenerator::visitCaseNode(CaseNode* node){
    INPUT : Node case.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitWhileNode(WhileNode* node){
    INPUT : Node while.
    OUTPUT : Instruksi loop.
    CARA KERJA :
    1. Simpan alamat awal.
    2. Generate kondisi.
    3. Emit JPC placeholder.
    4. Generate loop block.
    5. Emit JMP ke awal.
    6. Patch alamat akhir.
}

void CodeGenerator::visitRepeatNode(RepeatNode* node){
    INPUT : Node repeat.
    OUTPUT : Instruksi loop.
    CARA KERJA :
    1. Simpan alamat awal.
    2. Generate semua statement.
    3. Generate kondisi.
    4. Emit JPC kembali ke awal.
}

void CodeGenerator::visitForNode(ForNode* node){
    INPUT : Node for.
    OUTPUT : Instruksi loop.
    CARA KERJA :
    1. Generate nilai awal dan simpan ke iterator.
    2. Simpan alamat awal.
    3. Load iterator dan nilai akhir.
    4. Emit compare LEQ atau GEQ.
    5. Emit JPC placeholder.
    6. Generate body.
}

```

```

    7. Increment/decrement iterator.
    8. Emit JMP ke awal.
    9. Patch alamat akhir.
}

void CodeGenerator::visitProcCallNode(ProcCallNode* node){
    INPUT : Node pemanggilan prosedur.
    OUTPUT : Instruksi panggilan / I/O.
    CARA KERJA :
    1. Jika write/writeln, handle literal string/char sebagai OPR teks.
    2. Untuk argumen biasa, generate arg dan emit WRT.
    3. Jika writeln, emit WRTLN.
    4. Jika prosedur lain, generate argumen.
    5. Lookup entry address dan emit CAL.
}

void CodeGenerator::visitVarNode(VarNode* node){
    INPUT : Node variabel.
    OUTPUT : Instruksi load literal/variabel.
    CARA KERJA :
    1. Jika node punya tabIndex/lev, load langsung.
    2. Handle true/false sebagai LIT 1/0.
    3. Jika perlu, lookup symbol table lalu emit LOD.
}

void CodeGenerator::visitArrayAccessNode(ArrayAccessNode* node){
    INPUT : Node akses array.
    OUTPUT : Instruksi indeks dan base.
    CARA KERJA :
    1. Jika isLValueMode, hanya generate indeks.
    2. Jika r-value, generate base lalu semua indeks.
}

void CodeGenerator::visitRecordAccessNode(RecordAccessNode* node){
    INPUT : Node akses record.
    OUTPUT : Instruksi akses field.
    CARA KERJA :
    1. Jika isLValueMode, tidak melakukan apa-apa.
    2. Jika r-value, generate recordVar.
}

void CodeGenerator::visitBinOpNode(BinOpNode* node){
    INPUT : Node operasi biner.
    OUTPUT : Instruksi operasi aritmatika/logika.
    CARA KERJA :
    1. Generate operand kiri dan kanan.
    2. Emit OPR sesuai mapping operator.
}

```



```

void CodeGenerator::visitUnaryOpNode(UnaryOpNode* node){
    INPUT : Node operasi unary.
    OUTPUT : Instruksi unary.
    CARA KERJA :
    1. Jika not, generate operand lalu bandingkan dengan 0.
    2. Jika unary minus, generate operand lalu emit NEG.

void CodeGenerator::visitNumberNode(NumberNode* node){
    INPUT : Node angka.
    OUTPUT : Instruksi literal.
    CARA KERJA :
    1. Konversi string ke int.
    2. Emit LIT dengan nilai angka.
}

void CodeGenerator::visitStringNode(StringNode* node){
    INPUT : Node string.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

void CodeGenerator::visitCharNode(CharNode* node){
    INPUT : Node char.
    OUTPUT : Tidak menghasilkan instruksi.
    CARA KERJA :
    1. Diabaikan (void).
}

```

3. Stack.hpp & Stack.cpp

Stack.hpp

```

struct CallFrame {
    std::size_t returnAddress = 0;
    std::size_t lexicalLevel = 0;
    std::size_t baseAddress = 0;
    std::size_t staticLink = 0;
};

struct Cell {
    int value = 0;
    bool readOnly = false;
};

class Stack {
public:

```

```

void reset();

void push(int value);
int pop();
int peek(std::size_t depth = 0) const;
std::size_t stackSize() const;

void allocate(std::size_t count);
void store(std::size_t address, int value);
int load(std::size_t address) const;
void setReadOnly(std::size_t address, bool readOnly);
std::size_t dataSize() const;

std::size_t resolveAbsoluteAddress(int levelDiff, int tabIndex)
const;
std::size_t computeStaticLink(int levelDiff) const;

void pushFrame(std::size_t returnAddress, std::size_t lexicalLevel
= 0, std::size_t staticLink = 0);
bool hasFrame() const;
std::size_t returnAddress() const;
std::size_t lexicalLevel() const;
void popFrame();

private:
    std::vector<int> operandStack;
    std::vector<Cell> dataMemory;
    std::vector<CallFrame> callFrames;

    void ensureDataIndex(std::size_t index);
};

```

Bagian ini mendefinisikan Stack sebagai mesin tumpukan operand, memori data, dan stack frame. Atribut utamanya berupa operandStack untuk nilai sementara, dataMemory untuk penyimpanan variabel, callFrames untuk konteks pemanggilan, dan CallFrame berisi returnAddress, lexicalLevel, baseAddress. Fungsi yang penting seperti push/pop/peek untuk operand, allocate/store/load untuk data, pushFrame/popFrame untuk call stack, dan resolveAbsoluteAddress untuk alamat berbasis level.

Stack.cpp

```

void Stack::reset(){
    INPUT : -
    OUTPUT : Stack bersih.
    CARA KERJA :
    1. Mengosongkan operand stack.

```

```

    2. Mengosongkan data memory.
    3. Mengosongkan call frames.
}

void Stack::push(int value){
    INPUT : Integer value.
    OUTPUT : Value masuk ke operand stack.
    CARA KERJA :
    1. Menambahkan value ke operandStack.
}

int Stack::pop(){
    INPUT : -
    OUTPUT : Nilai teratas operand stack.
    CARA KERJA :
    1. Jika kosong, lempar error.
    2. Ambil elemen terakhir.
    3. Hapus elemen dan kembalikan nilainya.
}

int Stack::peek(std::size_t depth){
    INPUT : Kedalaman dari top.
    OUTPUT : Nilai pada posisi depth.
    CARA KERJA :
    1. Validasi depth.
    2. Hitung indeks dari atas.
    3. Kembalikan nilainya.
}

std::size_t Stack::stackSize() const{
    INPUT : -
    OUTPUT : Ukuran operand stack.
    CARA KERJA :
    1. Mengembalikan operandStack.size().
}

void Stack::ensureDataIndex(std::size_t index){
    INPUT : Index data.
    OUTPUT : Data memory siap diakses.
    CARA KERJA :
    1. Jika index di luar size, resize dataMemory.
    2. Isi elemen baru dengan 0.
}

void Stack::allocate(std::size_t count){
    INPUT : Jumlah slot data.
    OUTPUT : Data memory diperbesar jika perlu.
    CARA KERJA :

```

```

    1. Ambil base address frame aktif.
    2. Hitung ukuran minimum yang dibutuhkan.
    3. Resize memory jika kurang.
}

void Stack::store(std::size_t address, int value){
    INPUT : Alamat dan nilai.
    OUTPUT : Nilai tersimpan di data memory.
    CARA KERJA :
    1. Pastikan address valid dengan ensureDataIndex.
    2. Simpan value di dataMemory[address].
}

int Stack::load(std::size_t address) const {
    INPUT : Alamat data.
    OUTPUT : Nilai pada alamat atau 0.
    CARA KERJA :
    1. Jika address di luar size, kembalikan 0.
    2. Kembalikan dataMemory[address].
}

void Stack::setReadOnly(std::size_t address, bool readOnly) {
    INPUT : Alamat data dan boolean readOnly
    OUTPUT : readOnly milik dataMemory sudah di-set sesuai boolean.
    CARA KERJA :
    1. Memanggil fungsi ensureDataIndex() untuk memastikan index dengan
       size memory.
    2. Set dataMemory[address].readOnly menjadi boolean readOnly;
}

std::size_t Stack::dataSize() const{
    INPUT : -
    OUTPUT : Ukuran data memory.
    CARA KERJA :
    1. Mengembalikan dataMemory.size().
}

void Stack::pushFrame(std::size_t returnAddress, std::size_t
lexicalLevel){
    INPUT : Return address dan level.
    OUTPUT : Frame baru ditambahkan.
    CARA KERJA :
    1. Ambil base = dataMemory.size().
    2. Push CallFrame baru ke callFrames.
}

bool Stack::hasFrame() const{
    INPUT : -

```

```

    OUTPUT : Status ada/tidak frame.
    CARA KERJA :
    1. Mengembalikan !callFrames.empty().
}

std::size_t Stack::returnAddress() const{
    INPUT : -
    OUTPUT : Return address frame terakhir.
    CARA KERJA :
    1. Jika kosong, kembalikan 0.
    2. Ambil returnAddress dari frame terakhir.
}

std::size_t Stack::lexicalLevel() const {
    INPUT : -
    OUTPUT : Lexical level frame terakhir.
    CARA KERJA :
    1. Jika kosong, kembalikan 0.
    2. Ambil lexicalLevel frame terakhir.
}

void Stack::popFrame(){
    INPUT : -
    OUTPUT : Frame terakhir dihapus.
    CARA KERJA :
    1. Ambil baseAddress frame terakhir.
    2. Pop frame dari callFrames.
    3. Resize dataMemory ke base lama.
}

std::size_t Stack::resolveAbsoluteAddress(int levelDiff, int tabIndex)
const{
    INPUT : Selisih level dan tab index.
    OUTPUT : Alamat absolut data.
    CARA KERJA :
    1. Jika tidak ada frame, kembalikan tabIndex.
    2. Hitung target level = current - levelDiff.
    3. Cari frame dengan level tersebut dari atas.
    4. Kembalikan baseAddress + tabIndex.
    5. Jika tidak ditemukan, fallback tabIndex.
}

```

4. VM.hpp & VM.cpp

VM.hpp

```

class VM {
public:

```

```

explicit VM(const std::vector<Instruction>& program,
            std::istream& input = std::cin,
            std::ostream& output = std::cout);

void run();
void reset();

bool halted() const;
std::size_t programCounter() const;

const Stack& getStack() const;
Stack& getStack();

private:
    const std::vector<Instruction>* program;
    Stack stack;
    std::istream* input;
    std::ostream* output;
    std::size_t pc;
    bool stopped;

    void execute(const Instruction& instruction);
};

```

Bagian ini mendefinisikan kelas VM sebagai mesin eksekusi P-Code. Atribut utamanya program sebagai daftar instruksi, stack untuk eksekusi, input/output untuk I/O, pc sebagai program counter, dan stopped sebagai status halt. Fungsi run() mengeksekusi instruksi, reset() mengulang state, halted() mengecek status berhenti, programCounter() membaca posisi, getStack() mengakses stack, dan execute() untuk interpretasi satu instruksi.

VM.cpp

```

VM::VM(const std::vector<Instruction>& program, std::istream& input,
std::ostream& output){
    INPUT : Program P-Code, stream input, stream output.
    OUTPUT : Objek VM terinisialisasi.
    CARA KERJA :
    1. Simpan alamat program, input, dan output.
    2. Set pc = 0 dan stopped = false.
}

void VM::reset(){
    INPUT : -
    OUTPUT : VM ke state awal.
    CARA KERJA :
    1. Reset stack.
}

```

```

    2. Set pc = 0.
    3. Set stopped = false.
}

bool VM::halted() const{
    INPUT : -
    OUTPUT : Status berhenti.
    CARA KERJA :
    1. Mengembalikan stopped.
}

std::size_t VM::programCounter() const{
    INPUT : -
    OUTPUT : Nilai pc.
    CARA KERJA :
    1. Mengembalikan pc.
}

const Stack& VM::getStack() const{
    INPUT : -
    OUTPUT : Referensi const stack.
    CARA KERJA :
    1. Mengembalikan stack.
}

Stack& VM::getStack(){
    INPUT : -
    OUTPUT : Referensi stack yang bisa dimodifikasi.
    CARA KERJA :
    1. Mengembalikan stack.
}

void VM::run(){
    INPUT : -
    OUTPUT : Eksekusi program sampai halt/EOF.
    CARA KERJA :
    1. Jika program null, lempar error.
    2. Set stopped = false.
    3. Loop selama belum stop dan pc valid.
    4. Ambil instruksi pada pc.
    5. Panggil execute().
}

void VM::execute(const Instruction& instruction){
    INPUT : Satu instruksi.
    OUTPUT : Efek pada stack, memori, atau pc.
    CARA KERJA :
    1. Switch berdasarkan instruction.op.

```

2. LIT push literal ke stack dan pc++.
 3. LOD load dari memory (alamat absolut) lalu push.
 4. STO pop value dan simpan ke memory.
 5. INT alokasi data memory.
 6. CAL push frame dan pindah pc.
 7. JMP set pc ke target.
 8. JPC pop kondisi, lompat jika 0.
 9. OPR jalankan operasi aritmatika/logika/I-0.
 10. RET kembali ke caller atau stop jika frame habis.
- }

BAB III

PENGUJIAN

PENGUJIAN

3.1 Test Case 1

Test Case berhasil karena semua deklarasi, assignment, akses array, akses record, enum, case, dan for sudah valid secara semantik.

INPUT

```

1  program testCase1;
2
3  const
4      PI = 3.14;
5      MAX = 100;
6      HURUF = 'A';
7      PESAN = 'Halo Dunia';
8
9  type
10     Hari = (Senin, Selasa, Rabu, Kamis, Jumat);
11     Batas = 1 .. 100;
12     Siswa = record
13         umur : integer;
14         nilai : real;
15         lulus : boolean
16     end;
17     DataSiswa = array [1..100] of Siswa;
18     Matriks = array [1..10] of array [1..10] of integer;
19
20  var
21     i, j, k : integer;
22     murid : Siswa;
23     kelas : DataSiswa;
24     papan : Matriks;
25     libur : Hari;
26
27  begin
28     i := +10;
29     j := -5;
30
31     murid.umur := 15;
32     murid.nilai := PI * 2.0;
33     kelas[1].lulus := true;
34     papan[1][2] := 100;
35     libur := Senin;
36
37     case i of
38         1, 2, 3 : k := 10;
39         4 : k := 20;
40         5 : k := 0
41     end;
42
43     for i := 1 to MAX do
44     begin
45         if i = 50 then
46             kelas[i].lulus := true
47         else
48             kelas[i].lulus := false;
49         end;
50     end.

```

OUTPUT

```
1  0 INT 0 410
2  1 LIT 0 10
3  2 OPR 0 1
4  3 STO 0 59
5  4 LIT 0 5
6  5 OPR 0 1
7  6 STO 0 60
8  7 LIT 0 15
9  8 LODA 0 62
10 9 LIT 0 0
11 10 OPR 0 2
12 11 STOI 0 0
13 12 LOD 0 42
14 13 LIT 0 2
15 14 OPR 0 4
16 15 LODA 0 62
17 16 LIT 0 1
18 17 OPR 0 2
19 18 STOI 0 0
20 19 LIT 0 1
21 20 LODA 0 65
22 21 LIT 0 1
23 22 LIT 0 0
24 23 OPR 0 3
25 24 OPR 0 2
26 25 LIT 0 2
27 26 OPR 0 2
28 27 STOI 0 0
29 28 LIT 0 100
30 29 LODA 0 365
31 30 LIT 0 1
32 31 LIT 0 0
33 32 OPR 0 3
34 33 OPR 0 2
35 34 LIT 0 2
36 35 LIT 0 0
37 36 OPR 0 3
38 37 OPR 0 2
39 38 STOI 0 0
40 39 LOD 0 46
```

```
41 40 ST0 0 465
42 41 LIT 0 1
43 42 ST0 0 59
44 43 LOD 0 59
45 44 LIT 0 100
46 45 OPR 0 12
47 46 JPC 0 75
48 47 LOD 0 59
49 48 LIT 0 50
50 49 OPR 0 7
51 50 JPC 0 61
52 51 LIT 0 1
53 52 LODA 0 65
54 53 LOD 0 59
55 54 LIT 0 0
56 55 OPR 0 3
57 56 OPR 0 2
58 57 LIT 0 2
59 58 OPR 0 2
60 59 STOI 0 0
61 60 JMP 0 70
62 61 LIT 0 0
63 62 LODA 0 65
64 63 LOD 0 59
65 64 LIT 0 0
66 65 OPR 0 3
67 66 OPR 0 2
68 67 LIT 0 2
69 68 OPR 0 2
70 69 STOI 0 0
71 70 LOD 0 59
72 71 LIT 0 1
73 72 OPR 0 2
74 73 ST0 0 59
75 74 JMP 0 43
76 75 RET
77
```

3.2 Test Case 2

Test Case berhasil karena pemanggilan procedure/function, tipe parameter, operasi boolean, operasi aritmatika, while, repeat, dan for sudah valid secara semantik.

INPUT

```
1 program testCase2;
2
3 var
4   x, y, z : integer;
5   status : boolean;
6   hasil : real;
7
8 procedure HitungMundur(start : integer; prefix : string);
9 var
10  counter : integer;
11 begin
12  counter := start;
13  while counter > 0 do
14  begin
15    counter := counter - 1;
16  end;
17 end;
18
19 function CekValid(a, b : integer; limit : real) : boolean;
20 var
21  temp : real;
22 begin
23  temp := (a + b) / 2.0;
24  if temp ≤ limit then
25    CekValid := true
26  else
27    CekValid := false;
28 end;
29
30 begin
31  x := 10;
32  y := 20;
33
34  HitungMundur(x, 'Mulai: ');
35
36  status := not CekValid(x, y, 15.5) and (x <> y) or (x = 10);
37
38  repeat
39    z := x * y div 2 mod 3;
40    x := x + 1;
41  until x ≥ 20;
42
43  for z := 10 downto 1 do
44  begin
45    y := y - 1;
46  end;
47 end.
```

OUTPUT

1	0	INT	0	8
2	1	JMP	0	16
3	2	INT	0	4
4	3	STO	0	48
5	4	LOD	0	48
6	5	STO	0	50
7	6	LOD	0	50
8	7	LIT	0	0
9	8	OPR	0	11
10	9	JPC	0	15
11	10	LOD	0	50
12	11	LIT	0	1
13	12	OPR	0	3
14	13	STO	0	50
15	14	JMP	0	6
16	15	RET		
17	16	JMP	0	37
18	17	INT	0	4
19	18	STO	0	54
20	19	STO	0	53
21	20	STO	0	52
22	21	LOD	0	52
23	22	LOD	0	53
24	23	OPR	0	2
25	24	LIT	0	2
26	25	OPR	0	5
27	26	STO	0	55
28	27	LOD	0	55
29	28	LOD	0	54
30	29	OPR	0	12
31	30	JPC	0	34
32	31	LIT	0	1
33	32	STO	1	51
34	33	JMP	0	36
35	34	LIT	0	0
36	35	STO	1	51
37	36	RET		
38	37	LIT	0	10
39	38	STO	0	42
40	39	LIT	0	20
41	40	STO	0	43
42	41	LOD	0	42
43	42	CAL	0	2
44	43	LOD	0	42
45	44	LOD	0	43
46	45	LIT	0	15
47	46	LIT	0	0
48	47	STO	0	51
49	48	CAL	0	17
50	49	LOD	0	51

51	50	LIT	0	0
52	51	OPR	0	7
53	52	LOD	0	42
54	53	LOD	0	43
55	54	OPR	0	8
56	55	OPR	0	15
57	56	LOD	0	42
58	57	LIT	0	10
59	58	OPR	0	7
60	59	OPR	0	16
61	60	STO	0	45
62	61	LOD	0	42
63	62	LOD	0	43
64	63	OPR	0	4
65	64	LIT	0	2
66	65	OPR	0	5
67	66	LIT	0	3
68	67	OPR	0	6
69	68	STO	0	44
70	69	LOD	0	42
71	70	LIT	0	1
72	71	OPR	0	2
73	72	STO	0	42
74	73	LOD	0	42
75	74	LIT	0	20
76	75	OPR	0	10
77	76	JPC	0	61
78	77	LIT	0	10
79	78	STO	0	44
80	79	LOD	0	44
81	80	LIT	0	1
82	81	OPR	0	10
83	82	JPC	0	92
84	83	LOD	0	43
85	84	LIT	0	1
86	85	OPR	0	3
87	86	STO	0	43
88	87	LOD	0	44
89	88	LIT	0	1
90	89	OPR	0	3
91	90	STO	0	44
92	91	JMP	0	79
93	92	RET		

3.3 Test Case 3

Test Case gagal karena v1, v2, dan v3 bertipe array, tetapi dipakai seperti nilai integer biasa: dibandingkan dengan $\leq$, dijumlahkan/dikalikan, dijadikan iterator for, dan diisi langsung dengan nilai integer/real.

INPUT

```
1  program testCase3;
2
3  const
4  |   BATAS = 4;
5
6  var
7  |   i, total : integer;
8  |   selesai : boolean;
9
10 begin
11 |   i := 1;
12 |   total := 0;
13 |   selesai := false;
14
15 |   while i ≤ BATAS do
16 |   begin
17 |       total := total + i * 2;
18 |       i := i + 1;
19 |   end;;
20
21 |   if total ≥ 20 then
22 |       selesai := true
23 |   else
24 |       selesai := false;
25
26 |   total := total + 0;
27 end.
```

OUTPUT


```
1  0 INT 0 6
2  1 LIT 0 1
3  2 STO 0 43
4  3 LIT 0 0
5  4 STO 0 44
6  5 LIT 0 0
7  6 STO 0 45
8  7 LOD 0 43
9  8 LIT 0 4
10 9 OPR 0 12
11 10 JPC 0 22
12 11 LOD 0 44
13 12 LOD 0 43
14 13 LIT 0 2
15 14 OPR 0 4
16 15 OPR 0 2
17 16 STO 0 44
18 17 LOD 0 43
19 18 LIT 0 1
20 19 OPR 0 2
21 20 STO 0 43
22 21 JMP 0 7
23 22 LOD 0 44
24 23 LIT 0 20
25 24 OPR 0 10
26 25 JPC 0 29
27 26 LIT 0 1
28 27 STO 0 45
29 28 JMP 0 31
30 29 LIT 0 0
31 30 STO 0 45
32 31 LOD 0 44
33 32 LIT 0 0
34 33 OPR 0 2
35 34 STO 0 44
36 35 RET
```

3.4 Test Case 4

Test Case gagal karena Point adalah type record, bukan subprogram/constructor, sehingga Point(0, 0) tidak valid. Selain itu, print belum dideklarasikan sebagai procedure.

INPUT

```
1  program testCase4;
2
3  type
4      Arah = (Utara, Timur, Selatan, Barat);
5      Point = record
6          x : integer;
7          y : integer;
8          aktif : boolean
9      end;
10
11  var
12      posisi, tujuan : Point;
13      langkah : integer;
14      arahSekarang : Arah;
15
16  begin
17      posisi.x := 0;
18      posisi.y := 0;
19      posisi.aktif := true;
20
21      tujuan.x := 3;
22      tujuan.y := 2;
23      tujuan.aktif := true;
24
25      arahSekarang := Timur;
26      langkah := 0;
27
28      while posisi.x < tujuan.x do
29          begin
30              posisi.x := posisi.x + 1;
31              langkah := langkah + 1;
32          end;;
33
34      if posisi.x = tujuan.x then
35          arahSekarang := Utara;
36      else
37          arahSekarang := Barat;
38      end;
39
40      while posisi.y < tujuan.y do
41          begin
42              posisi.y := posisi.y + 1;
43              langkah := langkah + 1;
44          end;;
45
46      langkah := langkah + 0;
47  end.
```

OUTPUT

1	0	INT	0	11
2	1	LIT	0	0
3	2	LODA	0	51
4	3	LIT	0	0
5	4	OPR	0	2
6	5	STOI	0	0
7	6	LIT	0	0
8	7	LODA	0	51
9	8	LIT	0	1
10	9	OPR	0	2
11	10	STOI	0	0
12	11	LIT	0	1
13	12	LODA	0	51
14	13	LIT	0	2
15	14	OPR	0	2
16	15	STOI	0	0
17	16	LIT	0	3
18	17	LODA	0	54
19	18	LIT	0	0
20	19	OPR	0	2
21	20	STOI	0	0
22	21	LIT	0	2
23	22	LODA	0	54
24	23	LIT	0	1
25	24	OPR	0	2
26	25	STOI	0	0
27	26	LIT	0	1
28	27	LODA	0	54
29	28	LIT	0	2
30	29	OPR	0	2
31	30	STOI	0	0
32	31	LOD	0	43
33	32	STO	0	58
34	33	LIT	0	0
35	34	STO	0	57
36	35	LODA	0	51
37	36	LIT	0	0
38	37	OPR	0	2
39	38	LODI	0	0
40	39	LODA	0	54
41	40	LIT	0	0
42	41	OPR	0	2
43	42	LODI	0	0
44	43	OPR	0	9
45	44	JPC	0	60
46	45	LODA	0	51
47	46	LIT	0	0
48	47	OPR	0	2
49	48	LODI	0	0
50	49	LIT	0	1
51	50	OPR	0	2
52	51	LODA	0	51
53	52	LIT	0	0
54	53	OPR	0	2
55	54	STOI	0	0
56	55	LOD	0	57
57	56	LIT	0	1
58	57	OPR	0	2
59	58	STO	0	57
60	59	JMP	0	35

```
61 60 LODA 0 51
62 61 LIT 0 0
63 62 OPR 0 2
64 63 LODI 0 0
65 64 LODA 0 54
66 65 LIT 0 0
67 66 OPR 0 2
68 67 LODI 0 0
69 68 OPR 0 7
70 69 JPC 0 73
71 70 LOD 0 42
72 71 STO 0 58
73 72 JMP 0 75
74 73 LOD 0 45
75 74 STO 0 58
76 75 LODA 0 51
77 76 LIT 0 1
78 77 OPR 0 2
79 78 LODI 0 0
80 79 LODA 0 54
81 80 LIT 0 1
82 81 OPR 0 2
83 82 LODI 0 0
84 83 OPR 0 9
85 84 JPC 0 100
86 85 LODA 0 51
87 86 LIT 0 1
88 87 OPR 0 2
89 88 LODI 0 0
90 89 LIT 0 1
91 90 OPR 0 2
92 91 LODA 0 51
93 92 LIT 0 1
94 93 OPR 0 2
95 94 STOI 0 0
96 95 LOD 0 57
97 96 LIT 0 1
98 97 OPR 0 2
99 98 STO 0 57
100 99 JMP 0 75
101 100 LOD 0 57
102 101 LIT 0 0
103 102 OPR 0 2
104 103 STO 0 57
105 104 RET
```

3.5 Test Case 5

Test Case berhasil karena array diakses melalui indeks terlebih dahulu, sehingga assignment dilakukan ke elemen array bertipe integer, bukan ke array secara langsung.

INPUT

```
1  { state skip: brace comment }
2  (*
3  state skip: paren-star comment
4  state skip: paren-star comment
5  *)
6
7  program testCase5;
8  const
9  |   c1 = 123;
10 |   c2 = 123;
11 type
12 |   hama = integer;
13 |   t1 = hama;
14 var
15 |   v1, v2, v3: array[1..3] of integer;
16 |   i: integer;
17
18 function f1(a1: integer): integer;
19 begin
20 |   if a1 ≥ 10 then
21 |   |   a1 := a1 - 1
22 |   else
23 |   |   a1 := a1 + 2;
24 |   f1 := a1 div 2 mod 3;
25 end;
26
27 procedure p1;
28 begin
29 |   repeat
30 |   |   while v1[1] < v2[1] do
31 |   |   |   begin
32 |   |   |   |   v1[1] := v1[1] + 1
33 |   |   |   end;;
34 |   |
35 |   |   for i := 10 downto 1 do
36 |   |   |   begin
37 |   |   |   |   v2[1] := v2[1] * 2 div 4;
38 |   |   |   end;
39 |   until v1[1] = v2[1];
40 end;
41
42 begin
43 |   v1[1] := -42;
44 |   v2[1] := 3;
45 end.
46
```

OUTPUT

1	0	INT	0	8
2	1	JMP	0	16
3	2	INT	0	4
4	3	STO	0	48
5	4	LOD	0	48
6	5	STO	0	50
7	6	LOD	0	50
8	7	LIT	0	0
9	8	OPR	0	11
10	9	JPC	0	15
11	10	LOD	0	50
12	11	LIT	0	1
13	12	OPR	0	3
14	13	STO	0	50
15	14	JMP	0	6
16	15	RET		
17	16	JMP	0	37
18	17	INT	0	4
19	18	STO	0	54
20	19	STO	0	53
21	20	STO	0	52
22	21	LOD	0	52
23	22	LOD	0	53
24	23	OPR	0	2
25	24	LIT	0	2
26	25	OPR	0	5
27	26	STO	0	55
28	27	LOD	0	55
29	28	LOD	0	54
30	29	OPR	0	12
31	30	JPC	0	34
32	31	LIT	0	1
33	32	STO	1	51
34	33	JMP	0	36
35	34	LIT	0	0
36	35	STO	1	51
37	36	RET		
38	37	LIT	0	10
39	38	STO	0	42
40	39	LIT	0	20
41	40	STO	0	43
42	41	LOD	0	42
43	42	CAL	0	2
44	43	LOD	0	42
45	44	LOD	0	43
46	45	LIT	0	15
47	46	LIT	0	0
48	47	STO	0	51
49	48	CAL	0	17
50	49	LOD	0	51

51	50	LOD	0	42
52	51	LOD	0	43
53	52	OPR	0	8
54	53	OPR	0	15
55	54	LOD	0	42
56	55	LIT	0	10
57	56	OPR	0	7
58	57	OPR	0	16
59	58	STO	0	45
60	59	LOD	0	45
61	60	OPR	0	17
62	61	OPR	0	14
63	62	LOD	0	42
64	63	LOD	0	43
65	64	OPR	0	4
66	65	LIT	0	2
67	66	OPR	0	5
68	67	LIT	0	3
69	68	OPR	0	6
70	69	STO	0	44
71	70	LOD	0	42
72	71	LIT	0	1
73	72	OPR	0	2
74	73	STO	0	42
75	74	LOD	0	42
76	75	LIT	0	20
77	76	OPR	0	10
78	77	JPC	0	62
79	78	LIT	0	10
80	79	STO	0	44
81	80	LOD	0	44
82	81	LIT	0	1
83	82	OPR	0	10
84	83	JPC	0	93
85	84	LOD	0	43
86	85	LIT	0	1
87	86	OPR	0	3
88	87	STO	0	43
89	88	LOD	0	44
90	89	LIT	0	1
91	90	OPR	0	3
92	91	STO	0	44
93	92	JMP	0	80
94	93	RET		

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa compiler Arion pada milestone ini telah berhasil dikembangkan hingga tahap intermediate code generation dan eksekusi program. AST yang telah divalidasi pada tahap sebelumnya dapat diterjemahkan menjadi instruksi P-Code seperti LIT, LOD, STO, INT, JMP, JPC, CAL, OPR, dan RET.

Code Generator berhasil menangani translasi ekspresi, assignment, pemanggilan prosedur, output, serta struktur kontrol seperti if, while, repeat, dan for menggunakan mekanisme jump. Selain itu, Virtual Machine berhasil mengeksekusi instruksi tersebut dengan bantuan Stack untuk menyimpan operand, data memory, dan call frame.

Secara keseluruhan, implementasi ini menunjukkan bahwa bahasa Arion tidak hanya dapat dianalisis secara sintaksis dan semantik, tetapi juga sudah dapat diterjemahkan dan dijalankan hingga menghasilkan output program.

4.2. Saran

Untuk pengembangan selanjutnya, fitur code generation masih dapat diperluas, terutama pada struktur case, array, record, parameter subprogram, dan return value fungsi. Selain itu, error handling runtime dapat diperkuat dengan validasi seperti invalid jump target, akses memori di luar batas, overflow/underflow numerik, serta pengecekan stack frame yang lebih ketat.

Pengujian juga perlu diperluas dengan berbagai skenario program Arion agar hasil intermediate code dan eksekusi Virtual Machine dapat dipastikan berjalan konsisten sesuai semantik bahasa.

BAB V

LAMPIRAN

Link Repository Github: <https://github.com/staplesmaster/SSS-Tubes-IF2224-2026>

Pembagian Tugas:

Nama	NIM	Bagian	Persentase
Geraldo Artemius	13524005	• Laporan • Code Generator	20%
Mikhael Andrian Yonatan	13524051	• VM • Laporan	20%
Junior Natra Situmorang	13524055	• Laporan • Code Generation • Instruction • Stack • VM	20%
Reynard Nathanael	13524103	• Laporan • Stack	20%
Nicholas Luis Chandra	13524105	• Laporan • Instruction	20%

BAB VI

REFERENSI

<https://www.geeksforgeeks.org/compiler-design/intermediate-code-generation-in-compiler-design/>

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)

<https://www.geeksforgeeks.org/compiler-design/three-address-code-compiler/>